

Overview of Flow-based generative model

Subject: Flow-based generative model

1.

$$z_0 = F^{-1}(x) = z_T + \int_T^0 f(z_t, t) dt = z_T - \int_0^T f(z_t, t) dt \quad \{\displaystyle z_0 = F^{-1}(x) = z_T + \int_T^0 f(z_t, t) dt = z_T - \int_0^T f(z_t, t) dt\}$$

2.

$$p_1(z_1) = p_0(z_0) \left| \det \left(\frac{df_1(z_0)}{dz_0} \right) \right|^{-1} \quad \{\displaystyle p_1(z_1) = p_0(z_0) \left| \det \left(\frac{df_1(z_0)}{dz_0} \right) \right|^{-1}\}$$

3.

Learning probability distributions by differentiating such log Jacobians originated in the Infomax (maximum likelihood) approach to ICA, which forms a single-layer ($K=1$) flow-based model. Relatedly, the single layer precursor of conditional generative flows appeared in. To efficiently compute the log likelihood, the functions $f_1, \dots, f_K$ $\{\displaystyle f_1, \dots, f_K\}$ should be easily invertible, and the determinants of their Jacobians should be simple to compute. In practice, the functions $f_1, \dots, f_K$ $\{\displaystyle f_1, \dots, f_K\}$ are modeled using deep neural networks, and are trained to minimize the negative log-likelihood of data samples from the target distribution. These architectures are usually designed such that only the forward pass of the neural network is required in both the inverse and the Jacobian determinant calculations. Examples of such architectures include NICE, RealNVP, and Glow.

4.

$$R_{\text{cal}} \Delta(p; a, c) = \left| \det \left(\mathbf{R}_F \mathbf{p} \mathbf{E} \right) \right| = |a|^{1-n} \prod_{i=1}^n q_i p_i \quad \{\displaystyle R_{\text{cal}} \Delta(p; a, c) = \left| \det \left(\mathbf{R}_F \mathbf{p} \mathbf{E} \right) \right| = |a|^{1-n} \prod_{i=1}^n q_i p_i\}$$

5.

$\mathbf{E} = [\mathbf{I}_{n-1} \mathbf{1}]$ $\{\displaystyle \mathbf{E} = [\mathbf{I}_{n-1} \mathbf{1}]\}$. To define $\mathbf{U}$ $\{\displaystyle \mathbf{U}\}$, the differential volume element at the transformation input ($\mathbf{p} \in \Delta^{n-1}$ $\{\displaystyle \mathbf{p} \in \Delta^{n-1}\}$), we start with a rectangle in $\tilde{\mathbf{p}}$ $\{\displaystyle \tilde{\mathbf{p}}\}$ -space, having (signed) differential side-lengths, $dp_1, \dots, dp_{n-1}$ $\{\displaystyle dp_1, \dots, dp_{n-1}\}$ from which we form the square diagonal matrix $\mathbf{D}$ $\{\displaystyle \mathbf{D}\}$, the columns of which span the rectangle. At very small scale, we get $\mathbf{U} = e(\mathbf{D}) = \mathbf{E} \mathbf{D}$ $\{\displaystyle \mathbf{U} = e(\mathbf{D}) = \mathbf{E} \mathbf{D}\}$, with:

6.

Training method As is generally done when training a deep learning model, the goal with normalizing flows is to minimize the Kullback–Leibler divergence between the model's likelihood and the target distribution to be estimated. Denoting p_θ $\{\displaystyle p_\theta\}$ the model's likelihood and p^* $\{\displaystyle p^*\}$ the target distribution to learn, the (forward) KL-divergence is:

7.

To understand the geometric interpretation of the factor $\sqrt{n}$, see the example for the 1-simplex in the diagram at right. The differential volume element at the transformation output ($\mathbf{q} \in \Delta^{n-1}$), is the parallelotope, $V = f(\mathbf{U}) = \int \mathbf{F} \mathbf{p} \mathbf{E} \mathbf{D} / \{ \displaystyle V=f(\mathbf{U})=\wedge \mathbf{F}_{\{p\}} \mathbf{E} \mathbf{D} \} / \}$, where $\mathbf{F}_{\{p\}}$ is the n -by- n Jacobian of f at $\mathbf{p} = \mathbf{e}(\mathbf{p})$. Its volume is:

8.

$R_{\text{gcal}} \Delta(\mathbf{p}; \mathbf{A}, \mathbf{c}) = |\det \mathbf{A}| |\lambda| \prod_{i=1}^n q_i p_i \{ \displaystyle R_{\text{gcal}}^{\Delta}(\mathbf{p}; \mathbf{A}, \mathbf{c}) = \frac{|\det \mathbf{A}|}{|\lambda|} \prod_{i=1}^n \frac{q_i}{p_i} \}$

9.

$x = f_{\text{trans}}^{-1}(y; c) = \mathbf{y} - c$, where $\mathbf{y} = \mathbf{y}'c + (\mathbf{y}'c)^2 + 1 - c'c$
 $= f_{\text{trans}}^{-1}(\mathbf{y}; c) = \mathbf{y} - \mathbf{c}$
 $\text{where } \mathbf{y} = \mathbf{y} + \sqrt{(\mathbf{y} - \mathbf{c})^2 + 1 - \mathbf{c} \cdot \mathbf{c}}$

10.

Nonlinear Independent Components Estimation (NICE) Let $\mathbf{x}, \mathbf{z} \in \mathbb{R}^{2n}$ be even-dimensional, and split them in the middle. Then the normalizing flow functions are $\mathbf{x} = [\mathbf{x}_1 \mathbf{x}_2] = \mathbf{f}_{\theta}(\mathbf{z}) = [\mathbf{z}_1 \mathbf{z}_2] + [0 \mathbf{m}_{\theta}(\mathbf{z}_1)]$
 $\mathbf{x} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{bmatrix} = \mathbf{f}_{\theta}(\mathbf{z}) = \begin{bmatrix} \mathbf{z}_1 \\ \mathbf{z}_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \mathbf{m}_{\theta}(\mathbf{z}_1) \end{bmatrix}$
 $\mathbf{z} = \begin{bmatrix} \mathbf{z}_1 \\ \mathbf{z}_2 \end{bmatrix}$ where $\mathbf{m}_{\theta}$ is any neural network with weights θ .

11.

$f_{\text{gcal}}(\cdot; \mathbf{A}_1, \mathbf{c}_1) \mathbf{f}_{\text{gcal}}(\cdot; \mathbf{A}_2, \mathbf{c}_2) = f_{\text{gcal}}(\cdot; \mathbf{A}_1 \mathbf{A}_2, \mathbf{c}_1 + \mathbf{A}_1 \mathbf{c}_2)$
 $f_{\text{gcal}}(\cdot; \mathbf{A}_1, \mathbf{c}_1) \circ f_{\text{gcal}}(\cdot; \mathbf{A}_2, \mathbf{c}_2) = f_{\text{gcal}}(\cdot; \mathbf{A}_1 \mathbf{A}_2, \mathbf{c}_1 + \mathbf{A}_1 \mathbf{c}_2)$

12.

$R_f \Delta(\mathbf{p}) = \text{volume}(\mathbf{V}) / \text{volume}(\mathbf{U}) = |\det \mathbf{R}_{\mathbf{F}} \mathbf{p} \mathbf{E}| \{ \displaystyle R_f^{\Delta}(\mathbf{p}) = \frac{\text{volume}(\mathbf{V})}{\text{volume}(\mathbf{U})} = |\det \mathbf{R}_{\mathbf{F}} \mathbf{p} \mathbf{E}| \}$

13.

$p_1(z_1) = p_0(z_0) \left| \frac{df_1(z_0)}{dz_0} \right| - 1$
 $p_1(z_1) = p_0(z_0) \left| \frac{df_1(z_0)}{dz_0} \right|^{-1}$

14.

$$\mathbf{x} = \mathbf{f}^{-1}(\mathbf{y}; \mathbf{M}, \mathbf{c}) = \mathbf{M}^{-1}(\mathbf{y} - \mathbf{c}), \text{ where } \mathbf{y} = \mathbf{y}'\mathbf{W}\mathbf{c} + (\mathbf{y}'\mathbf{W}\mathbf{c})^2 + \mathbf{y}'\mathbf{W}\mathbf{y}(1 - \mathbf{c}'\mathbf{W}\mathbf{c})$$

$$\mathbf{x} = \mathbf{f}^{-1}(\mathbf{y}; \mathbf{M}, \mathbf{c}) = \mathbf{M}^{-1}(\mathbf{y} - \mathbf{c}), \text{ where } \mathbf{y} = \mathbf{y}'\mathbf{W}\mathbf{c} + (\mathbf{y}'\mathbf{W}\mathbf{c})^2 + \mathbf{y}'\mathbf{W}\mathbf{y}(1 - \mathbf{c}'\mathbf{W}\mathbf{c})$$

15.

Where $\det \frac{df_1^{-1}(z_1)}{dz_1}$ is the determinant of the Jacobian matrix of f_1^{-1} . By the inverse function theorem: